

End-Semester Examination Answer Sheet

CS633: Parallel Computing

Date: 4th May 2026

Time: 5 – 8 PM

Roll number 230316

Name CHAITANYA VISHNOI

(0 marks will be awarded if name and roll numbers are left empty)

PLEASE SWITCH OFF YOUR MOBILE PHONE AND KEEP IT IN YOUR BAG

- Write your name and roll number, your paper will not be evaluated if these are missing.
- Total time is 180 minutes. Total marks = 132.
- All questions are compulsory. There is no partial marking in programming questions.
- Write your name and roll number on rough sheets as well.
- You **must** answer all questions **on this sheet only** in the designated space. Violations lead to a penalty of -5. Ample space has been provided for every question. Think and be sure before you write your final answer on the answer sheet because question paper and extra sheets will not be collected for evaluation.
- **Legible** and neat answers within the designated area will be graded only; no marks will be awarded for illegible text.
- The MPI function prototypes are given on the last page of the question paper and the answer sheet.
- Read the questions carefully before answering. All clarifications must be directed to the instructor only.
- Assume that all uninitialized variables in the code (if any) are initialized to 0.
- Extra points will not be given for being verbose, so answer to the point, without being abrupt or confusing.
- Please do not leave the exam hall before we finish counting.
- **DO NOT OPEN THE QUESTION PAPER BEFORE THE SCHEDULED TIME. IF YOU ARE FOUND DOING SO, 5 MARKS WILL BE DEDUCTED AND YOU WILL LOSE 5 MINUTES.**

Marks

1a	1b	2a	2b	3a	3b	3c	3d	4a	4b	4c	5a
5	5	4	6	8	4	3	5	2	4	3	4
5b	5c	5d	6a	6b	7a	7b	8a	8b	9	10	Total
5	6	6	8	4	7	3	12	6	12	10	132

I pledge to answer all questions by myself without taking any external help.

Chaitanya Vishnoi (Sign your name)

A1-a.

0.001251
0.001579

A

0.023549
0.016614

C

0.009584
0.005939

B

1.093738
0.668975

F

0.243219
0.138770

D

0.756890
0.459700

E

A1-b.

The reason for differences in maxtime 1 and maxtime 2 is because of how blocking and ~~to~~ non-blocking collectives work.

For the command `mpirun -np 4 ./exe 2000 2`

Here since `arrsize = 2000`, we can safely say that both `allgather` and `Iallgather`'s ~~time~~ major cost source is latency as messages are short.

For two rounds, in first round cache and RAM are ~~not~~ firing up, and same goes for I/O. In second we compute the real time. ~~Where~~ Due to more communication rather than computation, we get $\text{maxtime1} < \text{maxtime2}$.

This is because there is no computation after the `Iallgather`, we directly call `waitall` instead this is not proper non-blocking, and for small `arrsize`, I/O communication is the main cost.

A2. (a)

In recursive ~~data~~ halving, number of ~~step~~ process half in every steps, and communication depends upon latency (L), Bandwidth (B) and computation at each step (C). For Total steps $\Rightarrow 2^k = P/S/K \approx \log_2 P$
 For $\log_2 P$ steps, $\boxed{\text{Cost} = \log_2 P * (L + \frac{D}{B} + D \cdot C)}$

A2. (b)

So, two ranks within ^{node} ~~rank~~ = intra-node, two ranks with same leaf = inter-node (1 hop)
 with different leaf = inter-switch (3 or 5 hops)
 Since there are $\log_2 16 = 4$ steps, each steps reduces the number of communication by half
 Step 0: ^⑥(0,1), ^⑥(2,3), ^⑥(4,5), ^⑥(6,7), ^⑥(8,9), ^⑥(10,11), ^⑥(12,13)
 $\text{vol} = n/2$ ^⑥(14,15), ~~(16,17)~~, all have ⑥ hops
 Step 1: ^①(0,2), ^②(4,6), ^①(8,10), ^①(12,14); $\text{vol} = n/4$ Max hop = 3, 4-6 (inter-switch)
 Step 2: ^①(0,4), ^③(8,12); $\text{vol} = n/8$ Max hop = 3, 8-12 inter-switch
 $\text{vol} = n/16$
 Step 3: ^③(0,8); Max hop = 3, 0-8 inter-switch
 Total hops bytes = $0 \times 8 + 3 \times 4 + 3 \times 2 + 3 \times 1 = 21$

A3-a) Fill in the blanks

```

1 // Assume all header files are included
2
3 *****
4
5 8
6
7 9 int main(int argc, char *argv[])
8 {
9
10 int numprocs, rank, level1_rank, level2_rank, local, i;
11 int arraySize = atoi(argv[1]); int ppn = atoi(argv[2]); float seed = atof(argv[3]);
12 double array[arraySize], *arrayRecv, *arrayAll, *arrayFinal;
13 double sum1=0.0, sum2=0.0, time_=0.0, time1=0.0, time2=0.0;
14 MPI_Comm level1_comm, level2_comm;
15
16
17 MPI_Init(&argc,&argv);
18 MPI_Comm_size(MPI_COMM_WORLD, &numprocs); // Assume numprocs%ppn is equal to 0
19 MPI_Comm_rank(MPI_COMM_WORLD, &rank);
20
21 srand (seed); //time(NULL));
22 for (i=0; i<arraySize; i++)
23     array[i] = (double) rand()/(((rank+1.0)*rand()))/(ppn*0.1));
24
25 // This is the non-hierarchical gather
26 if (rank == 0)
27     arrayRecv = (double *) malloc (arraySize*numprocs*sizeof(double));
28 time_ = MPI_Wtime ();
29 MPI_Gather (array, arraySize, MPI_DOUBLE, arrayRecv, arraySize, MPI_DOUBLE, 0, MPI_COMM_WORLD);
30 time_ = MPI_Wtime () - time_;
31 MPI_Reduce (&time_, &time1, 1, MPI_DOUBLE, MPI_MAX, 0, MPI_COMM_WORLD);
32
33 if (!rank) {
34     for (i=0; i<arraySize*numprocs; i++)
35         sum1 += arrayRecv[i];
36 }
37
38 // Write the hierarchical version of gather
39 // I. Form node-level sub-communicators (level1 groups)
40 int color = rank / (numprocs / 2);
41 MPI_Comm_split (MPI_COMM_WORLD, color, rank, &level1_comm);
42 MPI_Comm_rank (level1_comm, &level1_rank);
43
44 // II. Rank 'x' of above groups form a group, for example, rank 0 of the above groups form a group (level2 groups)
45 color = rank % (numprocs / 2) % - modulo
46 MPI_Comm_split (MPI_COMM_WORLD, color, rank, &level2_comm);
47 MPI_Comm_rank (level2_comm, &level2_rank);
48
49 if (level1_rank == 0)
50     arrayAll = (double *) malloc (arraySize*ppn*sizeof(double));
51
52 if (rank == 0)
53     arrayFinal = (double *) malloc (arraySize*numprocs*sizeof(double));
54
55 // Two-step gather (final result should be in arrayFinal array of rank 0 of MPI_COMM_WORLD)
56 time_ = MPI_Wtime ();

```

```

57 MPI_Gather ( array , arraySize , MPI_DOUBLE, arrayAll ,
arraySize/(numprocs/2) MPI_DOUBLE, 0 , level1_comm);
58
59 if ( level2-rank == 0 )
60 MPI_Gather ( arrayAll , arrayFinal , MPI_DOUBLE, arrayFinal ,
numprocs/2 , MPI_DOUBLE, 0 , level2_comm);
61
62 time_ = MPI_Wtime () - time_ ;
63 MPI_Reduce (&time_ , &time2, 1, MPI_DOUBLE, MPI_MAX, 0, MPI_COMM_WORLD);
64
65 if (!rank) {
66     for (i=0; i<arraySize*numprocs; i++)
67         sum2 += arrayFinal[i];
68 }
69 MPI_Comm_free (&level1_comm); MPI_Comm_free (&level2_comm);
70 MPI_Finalize();
71 }

```

A3-b)

- (A) time mpirun -np 48 --hosts host1:16,host2:16,host3:16 ./exe 10000 16 1000
 (B) time mpirun -np 48 --hosts host1:16,host2:16,host3:16 ./exe 1000 16 1000
 (C) time mpirun -np 16 ./exe 1000 16 1000
 (D) time mpirun -np 16 --hosts host1:4,host2:4,host3:4,host4:4 ./exe 1000 4 1000

0.04s 0.02s

0.007s 0.002s

0.001s 0.001s

0.0002s 0.0002s

A

B

D

C

A3-c)

- for case (C) : all processes are on same node, hence intra-node communication for array size only 1000 will be least.
- for case (D) : we still have 13 intra-node ~~and~~ but now 3 inter-node communications, hence time measured increases ~~slightly~~.
- for case (A) : Since, array size increases by 10x, bottle-neck at root comes into action, hence it has max time.

A3-d.

- (i) For each run the only function calls that cost us something are Comm-split & Gather.
- (ii) For run 3, we have least amount of array size and ~~least inter~~ ^{only 3 inter-node} with 13 intra-node communications, hence ~~there~~ it has least time measured, and
- (iii) Except run 3, Gather always takes more time than Comm-split because every process has to send data to root rank, hence communication time increases.
- (iv) Run 2 shows that huge array size creates bottleneck at root as now cost depends upon bandwidth than latency.
- (v) For ~~more~~ ^{huge} number of processes, even if N is constant, bandwidth ~~cost~~ increases linearly with $O(p)$, we see in run 4

A4-a.

No, the current code will not ~~the~~ give any output and will throw an error because the tag for Irecv is wrong, and is a mismatch.

A4-b.

① line 25: `if (recv(buf, arraysize, MPI_DOUBLE, rank+1, rankrank, rank, MPI_COMM_WORLD, request &request));`

The tag for receive was not corrected, and was ~~increased~~ incremented by 1.

line 27-28 ② The wait for `rank==0`, i.e. completion of receive for `rank==0` is called before `Isend`. In this case, the `rank==1` might not have sent anything yet i.e. its completion might be pending or overwritten. if (`rank==0`)
`MPI_wait (&request, &status);`
should be placed after line 34.

A4-c.

The ranks should be: $3 < 4 < 7 < 8$

→ The last rank has to be other than these all ranks communicate in intra-node manner and hence have, 12 microseconds less.

The rest are have to make inter-node communication

A5-a. Fill in the blanks

```

1 #include <stdio.h>
2 #include <string.h>
3 #include <stdlib.h>
4 #include <math.h>
5 #include "mpi.h"
6
7 int main (int argc, char *argv[])
8 {
9     int rank, size, root1, root2;
10    int color1, newrank1, newsize1, color2, newrank2, newsize2;
11    int sendbuf, recvbuf[2];
12    MPI_Comm newcomm1, newcomm2;
13
14    MPI_Init (&argc, &argv);
15    MPI_Comm_rank (MPI_COMM_WORLD, &rank);
16    MPI_Comm_size (MPI_COMM_WORLD, &size);    // Assume size is a perfect square
17
18    color1 = rank / size rank /  $\sqrt{\text{size}}$  ; // ranks of a row form a group
19    color2 = rank % size rank %  $\sqrt{\text{size}}$  ; // ranks of a column form a group
20
21    MPI_Comm_split (MPI_COMM_WORLD, color1, rank, &newcomm1);
22    MPI_Comm_rank (newcomm1, &newrank1); MPI_Comm_size (newcomm1, &newsize1);
23
24    MPI_Comm_split (MPI_COMM_WORLD, color2, rank, &newcomm2);
25    MPI_Comm_rank (newcomm2, &newrank2); MPI_Comm_size (newcomm2, &newsize2);
26
27    root1 = newsize1-1;
28    MPI_Reduce (&rank, &recvbuf[0], 1, MPI_INT, MPI_MAX, root1, newcomm1);
29    root2 = newsize2-1;
30    MPI_Reduce (&rank, &recvbuf[1], 1, MPI_INT, MPI_MAX, root2, newcomm2);
31
32    if (newrank1 == root1) printf ("%d 1 %d\n", rank, recvbuf[0]);
33    if (newrank2 == root2) printf ("%d 2 %d\n", rank, recvbuf[1]);
34
35    MPI_Comm_free(&newcomm1); MPI_Comm_free(&newcomm2);
36
37    MPI_Finalize();
38    return 0;
39 }

```

A5-b. What is the output of the command `mpirun -np 25 ./exe | sort -k1n`

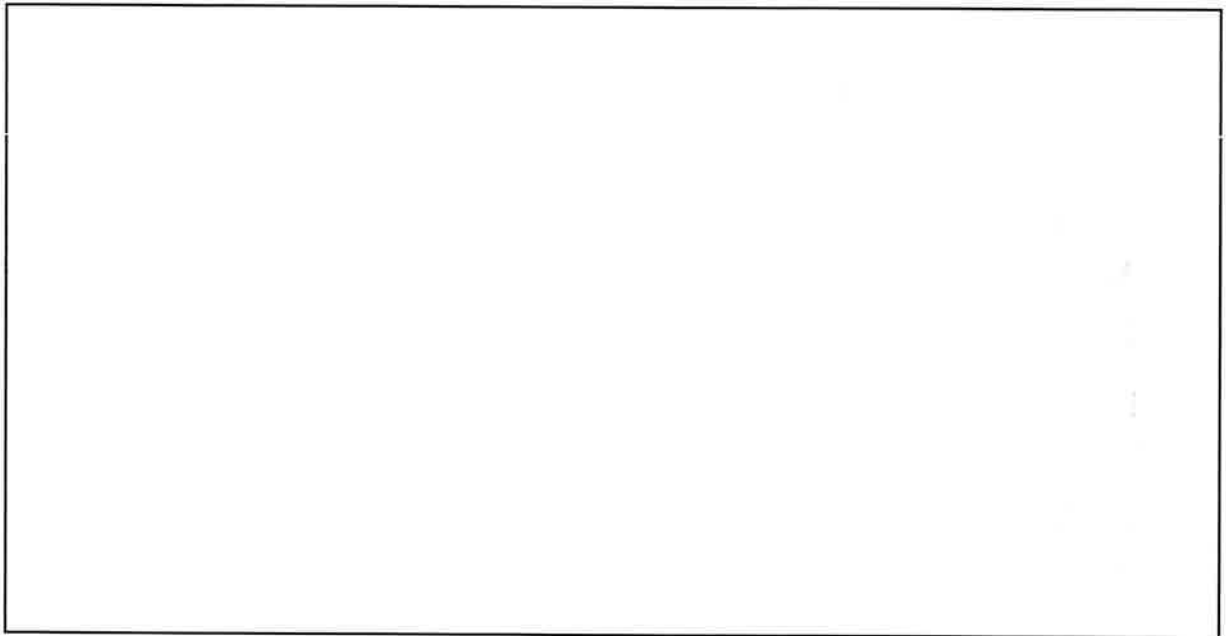
Use as many rows and columns as required for the output. Write the output in sorted order of the first column.

0	4				
0	20				
8	9				
1	21				
10	14				
2	22				
16	19				
3	23				
20	24				
4	24				

A5-c)

① Obviously, the cost would be lower for line 28 ^{but} ~~because~~ the hops would be lower and hence affecting final hop-byte.

② Intra-node communication will be more within Reduce of line 28.

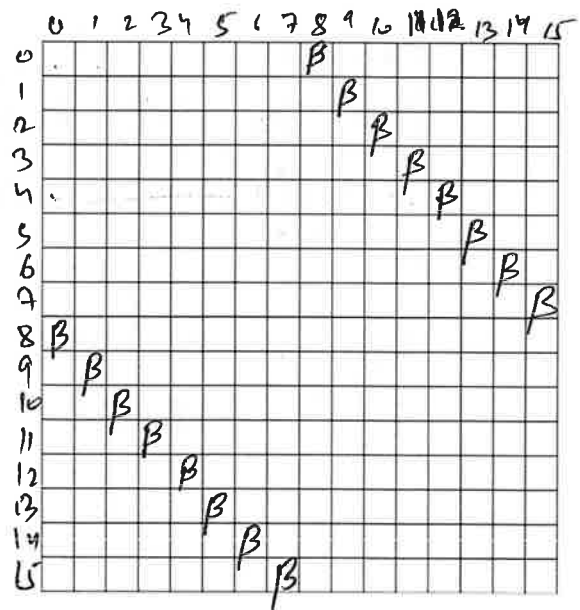
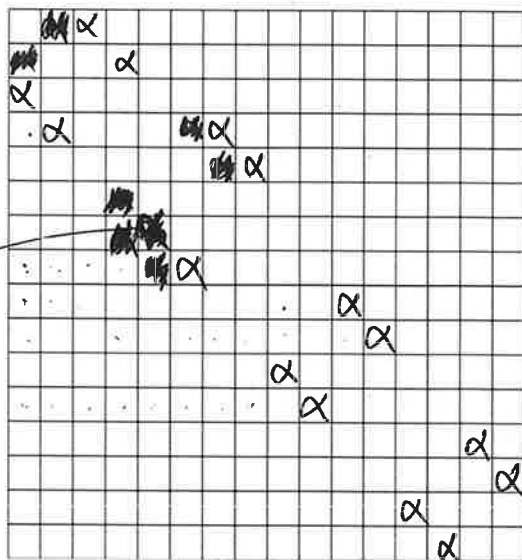


A5-d)

(i)

(ii)

$(7,5)_{\alpha}$



A6 (a) Complete the code.

```

1 // Sum all elements of two 2D arrays "data1" and "data2" (size: N x N)
2 // Assume 1D decomposition of the 2D domain along the y-axis, i.e. horizontal
3 // Assume N is divisible by size.
4 // Compute partial sum locally and then Reduce. Fill up the blanks in the code.
5
6 #include <stdlib.h>
7 #include <stdio.h>
8 #include "mpi.h"
9
10 int main (int argc, char *argv[])
11 {
12     int rank, size, root=0, sum[2] = {0}, N, recvbuf[2];
13     N = atoi(argv[1]);
14     int data1[N][N], data2[N][N];    // N is the global dimension
15
16     MPI_Init (&argc, &argv);
17     MPI_Comm_rank (MPI_COMM_WORLD, &rank);
18     MPI_Comm_size (MPI_COMM_WORLD, &size);
19
20     //initialize
21     for (int i=0; i<N; i++)
22         for (int j=0; j<N; j++)
23             data1[i][j] = i+j, data2[i][j] = i*j;
24
25     //Every rank computes partial sums of data1 and data2 (local sums may be stored in sum[0] and sum[1])
26     for (int i=0i=0; i < N; i++)
27         for (int j=0j=rank*size; j < N(rank+1)*size; j++) {
28             sum[0] += data1[i][j];
29             sum[1] += data2[i][j];
30         }
31
32     // Compute global sum of data1 and data2 and store in recvbuf
33     MPI_Reduce(&sum[0], &recvbuf[0], size, MPI_INT, MPI_SUM,
root, MPI_COMM_WORLD);
34
35     MPI_Finalize();
36     return 0;
37 }

```

A6-b.

So there are 2 processes and 2^8 columns from given matrix $N = 2^{10} \times 2^{10}$ on each node. Since 1D decomposition is of size 2^7 , we have 4 intra-node and 4 inter-node communications

A7-a. Complete the code.

```
1 // 0 sends last column of 2D array MxN to 1
2 // received in 1 contains 0's last column elements of data
3
4 #include <stdio.h>
5 #include <stdlib.h>
6 #include "mpi.h"
7
8 int main (int argc, char *argv[])
9 {
10  int myrank, size;
11  MPI_Status status;
12
13  MPI_Init (&argc, &argv);
14  MPI_Comm_rank (MPI_COMM_WORLD, &myrank);
15
16  MPI_Datatype newvtype;
17  int M = atoi (argv[1]) /*rows*/, N = atoi (argv[2]) /*columns*/;
18  int count = N;
19  int blocklen = 1;
20  int stride = M;
21  int data[M][N], received[count * blocklen];
22
23  MPI_Type_vector (count, blocklen, stride, MPI_INT, &newvtype);
24  MPI_Type_commit (&newvtype);
25
26  //initialize data
27  for (int i=0; i<M; i++)
28  for (int j=0; j<N; j++)
29  data[i][j]=0;
30
```

```

31 if (myrank == 0) /* code for process 0 */
32 {
33   for (int i=0; i<M; i++){
34     for (int j=0; j<N; j++){
35       data[i][j]=2*i+j; // 0 initializes
36     }
37
38     MPI_Send(&data[N-1], count*blocklen, newvtype, 1, 99, MPI_COMM_WORLD);
39   }
40 else if (myrank == 1) /* code for process 1 */
41 {
42   MPI_Recv(received, count*blocklen, MPI_INT, 0, 99, MPI_COMM_WORLD, &status);
43 }
44
45 MPI_Type_free (&newvtype);
46
47 MPI_Finalize();
48 return 0;
49
50 }
51

```

A7-b.

Write the content of "received" array at rank 1 for the execution command:

mpirun -np 2 ./exe 4 3

2 4 6 8

A8-a. Fill in the required blanks.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include "mpi.h"
4
5 int main (int argc, char *argv[])
6 {
7     int rank, size, recvcnt, index, numbytes, BUFSIZE;
8     char *filename="sharedFile";
9     MPI_Status status;
10
11     MPI_Init (&argc, &argv);
12     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
13     MPI_Comm_size(MPI_COMM_WORLD, &size);
14     int Dx = atoi (argv[1]), Dy = atoi (argv[2]) /*rows*/;
15     int Px = atoi (argv[3]), Py = atoi (argv[4]);
16     int data[Dy*Dx], read[Dy*Dx];
17
18     // initialize entire 2D domain data
19     for (int i=0; i<Dy*Dx; i++)
20         data[i]=rand()+i;
21
22     int rank_row = _____, rank_col = _____;
23
24     int start = _____; // Hint: use "rank_row" and "rank_col"
25     // Assume every process writes 'numbytes' bytes (BUFSIZE integers)
26
27     BUFSIZE = _____;
28     numbytes = BUFSIZE *sizeof(int);
29
30     MPI_File fh;
31     MPI_File_open (MPI_COMM_WORLD, filename, MPI_MODE_CREATE | MPI_MODE_RDWR, MPI_INFO_NULL, &fh);
32
33     MPI_Offset fo = (MPI_Offset) _____;
34
35     // File write using explicit offset (independent I/O)
36
37     MPI_File_close (&fh);
38     MPI_Finalize();
39     return 0;
40 }
```

A8-b.

(i)

(ii)

(iii)

A9.

Job ID	Submit time	Requested # nodes	Requested wall-clock time (hours)	Start time (a)	Wait time (hours) (b)
1	00:00	32	2	00:00	0
2	00:00	32	4	00:00	0
3	01:00	64	6	05:00	4
4	01:00	12	8	01:00	0
5	01:00	24	2	02:00	1
6	02:00	4	8	02:00	0
7	02:00	8	3	02:00	0
8	05:00	64	2	09:00	6

(c) Average wait time: $\frac{0 + 0 + 4 + 0 + 1 + 0 + 0 + 6}{8} = 1.375 \text{ hrs}$

total wait hours = 11 hrs

(d) Total idle node-hours: start = 00:00, end = 13:00

total active hours = 13 hrs

total ~~idle~~ idle hours = 11 hrs

(e) Backfill job IDs: 4, 6, 7

A10.

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include "mpi.h"
4
5 int main (int argc, char *argv[])
6 {
7     int myrank, size, recvcnt, index;
8     MPI_Status status;
9
10    MPI_Init (&argc, &argv);
11    MPI_Comm_rank(MPI_COMM_WORLD, &myrank);
12
13    // input: N: size of 3D array, idx: y-index of the xz-plane to be sent
14    int N = atoi (argv[1]);
15    int idx = atoi (argv[2]);
16
17    int data[N*N*N], received[ $N \times dx \times N$ ];
18
19    MPI_Datatype newvtype;
20    int count =  $N$ ;
21    int blocklen =  $N$ ;
22    int stride =  $dx$ ;
23
24    MPI_Type_vector (count, blocklen, stride, MPI_INT, &newvtype);
25    MPI_Type_commit (&newvtype);
26
27    //initialize data
28    for (int i=0; i<N; i++)
29        for (int j=0; j<N; j++)
30            for (int k=0; k<N; k++) {
31                data[ $i + j \times dx + k \times dx \times N$ ] = 0; // initialize all elements to 0
32            }
33
34    if (myrank == 0) /* code for process 0 */
35    {
36        for (int i=0; i<N; i++)
37            for (int j=0; j<N; j++)
38                for (int k=0; k<N; k++)
39                    data[ $i + j \times dx + k \times dx \times N$ ] = i*1+j*2+k*3; // some random initialization
40
41    //send to rank 1
42    MPI_Send (&data[ $N \times dx \times N$ ], 1, newvtype, 1, 99, MPI_COMM_WORLD);
43    }
44    else if (myrank == 1) /* code for process 1 */
45    {
46        MPI_Recv (received,  $N \times dx \times N$ , MPI_INT, 0, 99, MPI_COMM_WORLD, &status);
47    }
48
49    MPI_Type_free (&newvtype);
50    MPI_Finalize();
51    return 0;
52 }

```